

PYTHON AUTOMATION

LAB MANUAL

Cron Jobs with Python on Ubuntu/Linux

A practical, fully documented laboratory guide for scheduling Python automation tasks with Linux cron. The manual covers cron syntax, Python scripts, virtual environments, environment variables, logging, troubleshooting, email automation, best practices, and a final practical exercise.

Subject	Python Automation
Platform	Ubuntu / Linux
Level	Beginner → Intermediate
Practical focus	Scheduled Python execution

1. Lab Overview

Cron is a time-based job scheduler available on Unix-like operating systems. A cron job tells the operating system to execute a command at a defined time or interval. In Python automation, cron is commonly used to launch Python scripts that send reports, process files, call APIs, create backups, clean directories, or generate scheduled notifications.

The key distinction is simple: cron is an operating-system scheduler; Python contains the automation logic.

Learning Outcomes

- Explain what a cron job is and how it differs from a Python scheduler.
- Read and construct a five-field cron expression.
- Create a Python automation script suitable for scheduled execution.
- Install, edit, test, verify, and remove cron jobs.
- Run Python scripts using absolute paths and virtual environments.
- Capture standard output and errors in log files.
- Use environment variables safely for configuration.
- Troubleshoot common cron/Python execution problems.

2. Prerequisites

This laboratory assumes Ubuntu/Linux and Python 3.

Check Python

```
python3 --version
```

Check the cron service

```
systemctl status cron
```

Install cron if required

```
sudo apt update
sudo apt install cron
```

Enable and start it

```
sudo systemctl enable --now cron
```

3. Cron Architecture

```
Cron daemon
↓
Cron schedule
↓
Shell command
↓
Python interpreter
↓
Python automation script
↓
Output / files / API / email / database
```

Cron does not understand Python functions. It launches the command configured in the crontab. For reliability, use absolute paths because cron's execution environment is more limited than an interactive terminal.

Example

```
0 9 * * * /usr/bin/python3 /home/yousif/scripts/report.py
```

This means: run report.py at 09:00 every day.

4. Cron Expression Syntax

```
##### minute (0-59)
# ##### hour (0-23)
# # ##### day of month (1-31)
# # # ##### month (1-12)
# # # # ##### day of week (0-7; Sunday is 0 or 7)
# # # # #
* * * * * command
```

Field	Allowed values	Meaning
Minute	0–59	Minute within the hour
Hour	0–23	Hour using 24-hour time
Day of month	1–31	Calendar day
Month	1–12	January through December
Day of week	0–7	Sunday is 0 or 7; Monday is 1

Common operators

```
*          every allowed value
*/10       every 10 units
1-5        a range
1,15       specific values
```

5. Essential Cron Examples

```
# Every minute
* * * * * command

# Every 5 minutes
*/5 * * * * command

# Every day at 09:00
0 9 * * * command

# Every day at 18:30
30 18 * * * command

# Every Monday at 08:00
0 8 * * 1 command

# First day of every month at 09:00
0 9 1 * * command

# Weekdays at 17:00
0 17 * * 1-5 command
```

Cron timing uses the machine's configured local time zone. Verify the server clock and timezone when timing matters.

6. Practical 1 — Create a Python Automation

Create a laboratory directory:

```
mkdir -p ~/cron-lab
cd ~/cron-lab
```

Create hello_cron.py:

```
from datetime import datetime
from pathlib import Path

log_file = Path.home() / "cron-lab" / "execution.log"
current_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

with log_file.open("a", encoding="utf-8") as file:
    file.write(f"Cron executed at {current_time}\n")

print(f"Cron executed at {current_time}")
```

Code explanation

- `datetime.now()` obtains the current local date and time.
- `strftime()` converts it into a readable timestamp.
- `Path.home()` locates the current user's home directory.
- `open(..., 'a')` appends a new line without deleting earlier records.
- The `with` statement closes the file automatically.
- `print()` produces terminal output that can also be redirected by cron.

Test manually first:

```
python3 ~/cron-lab/hello_cron.py
cat ~/cron-lab/execution.log
```

Manual testing should come before scheduling. Cron should schedule a working program, not be used to discover basic application errors.

7. Practical 2 — Add the Script to Crontab

Open your user's crontab:

```
crontab -e
```

Add this temporary one-minute test entry, replacing YOUR_USERNAME:

```
* * * * * /usr/bin/python3 /home/YOUR_USERNAME/cron-lab/hello_cron.py
```

Wait about a minute and inspect the log:

```
cat ~/cron-lab/execution.log
```

Once verified, remove the test line with `crontab -e`.

8. Practical 3 — Redirect Output and Errors

Cron normally has no interactive terminal. Redirecting stdout and stderr makes failures observable.

```
* * * * * /usr/bin/python3 /home/YOUR_USERNAME/cron-lab/hello_cron.py >> /home/YOUR_USERNAME/cron-lab/cron.log 2>&1
```

`>>` appends standard output. `2>&1` sends standard error to the same log.

```
tail -n 20 ~/cron-lab/cron.log
```

9. Practical 4 — Use a Python Virtual Environment

Cron does not need to activate your shell environment if you call the environment's Python interpreter directly.

```
cd ~/cron-lab
python3 -m venv .env
source .env/bin/activate
python --version
deactivate
```

Use this interpreter from cron:

```
~/cron-lab/.env/bin/python
```

```
* /5 * * * * /home/YOUR_USERNAME/cron-lab/.env/bin/python /home/YOUR_USERNAME/cron-lab/hello_cron.py >> /home/YOUR_USERNAME/cron-lab/hello_cron.log
```

This is more deterministic than relying on `source .env/bin/activate`.

10. Practical 5 — Environment Variables and .env

Scheduled programs often require API keys, SMTP credentials, database URLs, and other configuration. Keep secrets outside the Python source code.

Install python-dotenv:

```
source ~/cron-lab/.env/bin/activate
pip install python-dotenv
deactivate
```

Example .env:

```
APP_NAME=CronAutomation
ENVIRONMENT=production
```

Python:

```
import os
from dotenv import load_dotenv

load_dotenv()
app_name = os.getenv("APP_NAME")
environment = os.getenv("ENVIRONMENT")

print(f"{app_name} is running in {environment} mode.")
```

Because cron's working directory may differ from your terminal, explicitly change into the project directory:

```
* /5 * * * * cd /home/YOUR_USERNAME/cron-lab && /home/YOUR_USERNAME/cron-lab/.env/bin/python /home/YOUR_USERNAME/cron-lab/hello_cron.py
```

Never commit real credentials. Add `.env` to `.gitignore`.

11. Practical 6 — Scheduled Email Automation

Cron is useful for scheduled email reports: cron determines when the script runs, while Python handles SMTP and HTML email construction.

Illustrative script:

```
import os
import smtplib
import ssl
from email.mime.text import MIMEText
from dotenv import load_dotenv

load_dotenv()

smtp_server = os.getenv("SMTP_SERVER")
smtp_port = int(os.getenv("PORT", "465"))
email_user = os.getenv("EMAIL")
```

```

email_password = os.getenv("PASSWORD")
recipient = os.getenv("RECIPIENT")

message = MIMEText(
    "<h2>Daily Automation Report</h2>"
    "<p>This message was generated automatically by Python and cron.</p>",
    "html"
)
message["Subject"] = "Daily Automation Report"
message["From"] = email_user
message["To"] = recipient

context = ssl.create_default_context()

with smtplib.SMTP_SSL(smtp_server, smtp_port, context=context) as server:
    server.login(email_user, email_password)
    server.send_message(message)

print("Email sent successfully.")

```

Example .env:

```

SMTP_SERVER=smtp.example.com
PORT=465
EMAIL=automation@example.com
PASSWORD=your-secret
RECIPIENT=recipient@example.com

```

Use the SMTP authentication method required by your provider. For Gmail, an app password may be required depending on the account's security configuration; do not assume the normal account password is accepted.

Daily 09:00 schedule:

```

0 9 * * * cd /home/YOUR_USERNAME/cron-lab && /home/YOUR_USERNAME/cron-lab/.venv/bin/python /home/YOUR_USERNAME/cron-l

```

12. Managing Crontab

List current jobs:

```

crontab -l

```

Edit jobs:

```

crontab -e

```

Remove the entire current user's crontab:

```

crontab -r

```

Use `crontab -r` carefully; it removes all jobs for that user.

Common system-wide locations:

```

/etc/crontab
/etc/cron.d/
/etc/cron.hourly/
/etc/cron.daily/
/etc/cron.weekly/
/etc/cron.monthly/

```

13. Cron Environment — A Critical Concept

A command that works in your terminal can fail under cron because `PATH`, the working directory, virtual-environment activation, and other environment variables may differ.

Prefer absolute executables and paths:

```
/home/YOUR_USERNAME/cron-lab/.venv/bin/python
/home/YOUR_USERNAME/cron-lab/email_report.py
```

Also use absolute paths for input/output files, or explicitly `cd` into the project directory.

14. Troubleshooting

Problem	Likely cause	Check
Job does not run	Cron service stopped	<code>systemctl status cron</code>
Python not found	Different PATH	Use absolute Python path
<code>ModuleNotFoundError</code>	Wrong interpreter	Use <code>.venv/bin/python</code>
File not found	Wrong working directory	Use absolute paths or <code>cd</code>
Permission denied	File/user permissions	<code>ls -l</code> ; check ownership
Works manually, fails in cron	Environment differs	Log <code>stdout/stderr</code>
No visible output	No interactive terminal	Redirect to log
Wrong time	Timezone mismatch	<code>date</code> ; <code>timedatectl</code>

Useful diagnostic commands

```
systemctl status cron
crontab -l
which python3
python3 --version
date
timedatectl
ls -l ~/cron-lab/
tail -f ~/cron-lab/cron.log
journalctl -u cron --since "30 minutes ago"
```

15. Practical 7 — Production-Style Scheduled Task

Create a task with explicit logging and exception handling:

```
from datetime import datetime
from pathlib import Path
import traceback

BASE_DIR = Path.home() / "cron-lab"
LOG_FILE = BASE_DIR / "automation.log"

def log(message):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with LOG_FILE.open("a", encoding="utf-8") as f:
        f.write(f"[{timestamp}] {message}\n")

def main():
    log("Automation started.")
    # Place real automation here:
    # API request, database task, report generation, email, etc.
    log("Automation completed successfully.")

if __name__ == "__main__":
    try:
        main()
    except Exception as exc:
        log(f"Automation failed: {exc}")
        log(traceback.format_exc())
```

raise

Schedule every 10 minutes:

```
* /10 * * * * cd /home/YOUR_USERNAME/cron-lab && /home/YOUR_USERNAME/cron-lab/.venv/bin/python /home/YOUR_USERNAME/cron-lab/src/automation.py
```

The design principle is: **cron decides when; Python decides what**. Keep business logic in Python rather than putting complex logic into the crontab.

16. Recommended Project Structure

```
cron-automation/
├── .env
├── .gitignore
├── README.md
├── requirements.txt
├── .venv/
├── src/
│   ├── automation.py
│   ├── logs/
│   ├── automation.log
│   └── scripts/
│       └── run.sh
```

For larger automations, a shell wrapper can establish the working directory and launch the correct Python interpreter:

```
#!/bin/bash
set -e
cd /home/YOUR_USERNAME/cron-automation
exec /home/YOUR_USERNAME/cron-automation/.venv/bin/python /home/YOUR_USERNAME/cron-automation/src/automation.py

chmod +x ~/cron-automation/scripts/run.sh

0 9 * * * /home/YOUR_USERNAME/cron-automation/scripts/run.sh >> /home/YOUR_USERNAME/cron-automation/logs/cron.log 2>&
```

17. Best Practices

- Use absolute paths in cron commands.
- Use the project's exact Python interpreter.
- Log stdout and stderr for important jobs.
- Keep secrets outside source code.
- Test manually before scheduling.
- Keep cron entries simple and Python logic in Python.
- Make tasks idempotent where possible.
- Handle exceptions and transient failures deliberately.
- Avoid unnecessary high-frequency schedules.
- Document production cron entries.

18. Cron Compared with Other Scheduling Options

Tool	Best suited for
Cron	Simple time-based Linux jobs
Python schedule	Simple schedules inside a long-running Python process

APScheduler	Advanced Python scheduling and job management
Celery + Celery Beat	Distributed background tasks and scheduled workloads
systemd timers	Linux-native service-oriented scheduling

For beginner Linux-based Python automation, cron is a strong starting point because its scheduling model is simple and widely supported.

19. Final Laboratory Exercise

Build a scheduled Python automation that runs once every day at 09:00. It must record its execution time, load configuration from `.env`, write a log entry, and produce a clear success or failure message. Redirect cron output to a log file.

Required deliverables

- Working Python script.
- Virtual environment with required dependencies.
- `.env` containing configuration values; never submit real credentials.
- `.gitignore` excluding `.env` and the virtual environment.
- Crontab entry scheduled for 09:00.
- Log proving successful execution.
- Short explanation of the cron expression.

Suggested cron entry

```
0 9 * * * cd /home/YOUR_USERNAME/cron-lab && /home/YOUR_USERNAME/cron-lab/.venv/bin/python /home/YOUR_USERNAME/cron-l
```

20. Viva / Review Questions

1. What is a cron job?
2. What is the role of the cron daemon?
3. Is cron a Python library?
4. What are the five scheduling fields?
5. What does `*/5` mean?
6. How do you schedule a script every day at 20:30?
7. Why use absolute paths?
8. Why can a module be available in the terminal but missing under cron?
9. How do you use a virtual environment from cron?
10. What does `>>` do?
11. What does `2>&1` do?
12. How do you list the current user's jobs?
13. How do you edit the crontab?
14. Why should secrets not be hard-coded?

15. Why is logging important?

16. What is the difference between cron and a Python scheduler?

21. Quick Reference Sheet

```
# View jobs
crontab -l

# Edit jobs
crontab -e

# Check cron
systemctl status cron

# Enable/start
sudo systemctl enable --now cron

# Every minute
* * * * * command

# Every 5 minutes
*/5 * * * * command

# Daily at 09:00
0 9 * * * command

# Weekdays at 17:00
0 17 * * 1-5 command

# Run a venv Python
/home/user/project/.venv/bin/python /home/user/project/app.py

# Log stdout + stderr
command >> /path/to/cron.log 2>&1

# Inspect logs
tail -n 50 /path/to/cron.log
tail -f /path/to/cron.log
```

Core principle: **cron decides when; Python decides what.** Once this distinction is clear, cron becomes a simple and powerful foundation for Linux-based Python automation.

End of Lab Manual